Packed in-memory SST index for VDisk

-75%

LogoBlob SST index memory on production snapshots
43.9 → 10.9 bytes per record

ydbd RSS in an end-to-end test
2.3x smaller

on-disk format
unchanged

price
CPU: build + decode

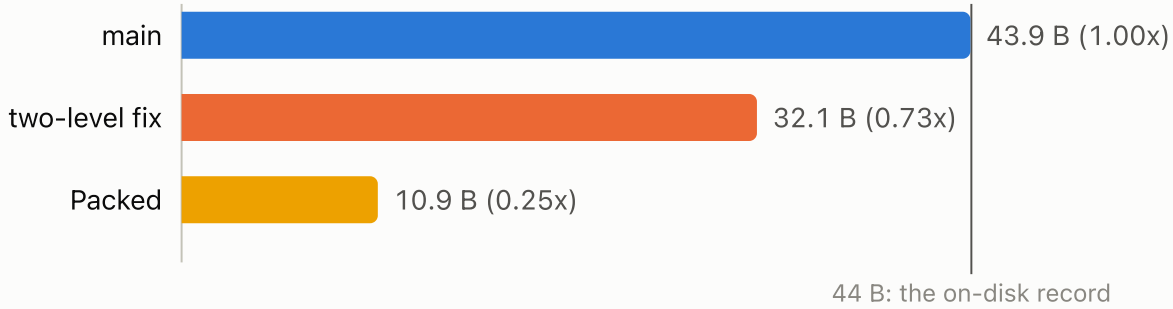
default
off: EnableVDiskPackedSstIndex

ydb-platform/ydb PR #56155 · VDisk / hull: LogoBlob SST index forms · measured Oct 2026

Spend CPU to save memory

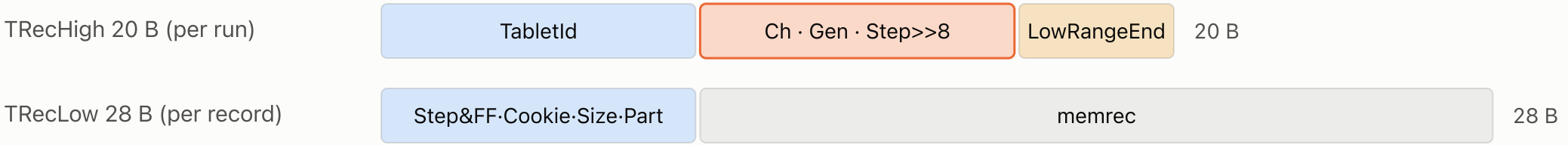
- Every LogoBlob SST keeps its index in RAM for its lifetime; it grows linearly with the data a VDisk holds.
- The on-disk record is **44 B** (24 B key + 20 B memrec). On main the in-memory index takes **43.9 B/record** on production data: as much as the disk format.
- This PR: a **two-level fix** as the default form, and the compact **Packed** form behind a feature flag, chosen per SST when it is loaded or written by compaction.
- Disk format and everything above the segment stay unchanged.

Index bytes per record, all 5 snapshots

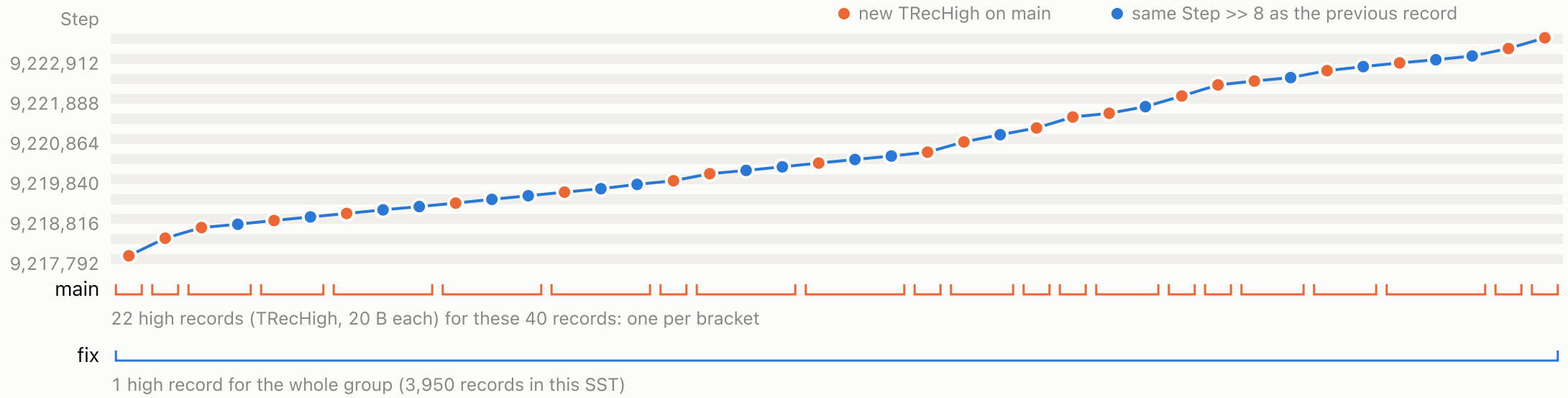


Totals: main 4,596 MiB, two-level fix 3,353 MiB, Packed 1,144 MiB for 109.7 M records. 5 production index snapshots of block-storage clusters (block-4-2), 4,212 SSTs, 109.7 M records; memrecs synthesised.

The shared high part holds Step >> 8, which changes every few records

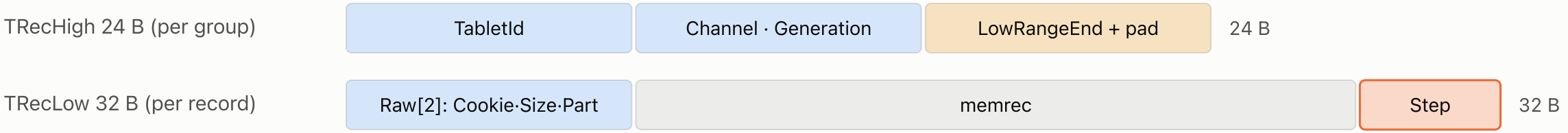


Step of 40 consecutive records of one tablet / channel / generation group in the largest SST of one snapshot. Block-storage tablets advance Step per commit; each shaded band is one value of Step >> 8.

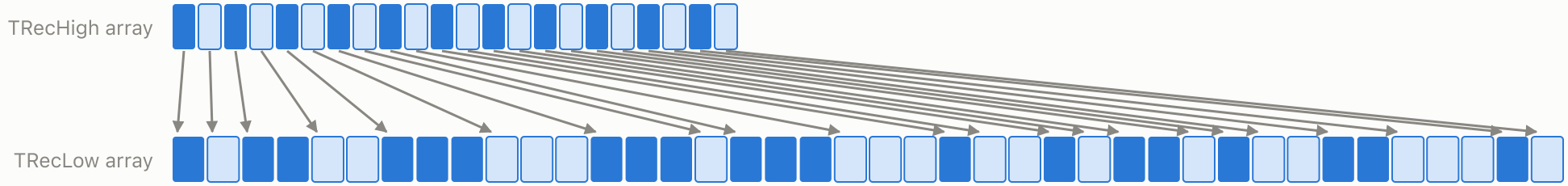


A new 20-byte high record starts whenever Step >> 8 changes: here every 1.8 records, over the whole SST every 1.05, over all snapshots every 1.25. So 28 + 20 / 1.25 = 44 B per record (measured 43.9): as big as the flat on-disk record.

Step moves into the low part: one high record per (tablet, channel, generation)

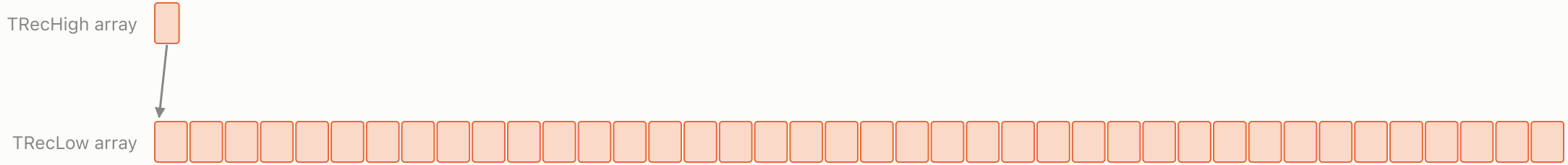


main: a new TRecHigh (20 B) whenever Step >> 8 changes: 22 for these 40 records



22 x 20 B + 40 x 28 B = 1,560 B: 39.0 B per record here; 47 in the whole SST, 43.9 over all snapshots

fix: one TRecHigh (24 B) per (tablet, channel, generation) group



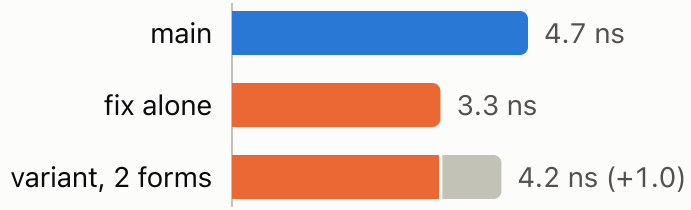
1 x 24 B shared by the group's 3,950 records + 32 B per record: 32.0 B per record (32.05 over all snapshots)

... and 3,910 more records of this group

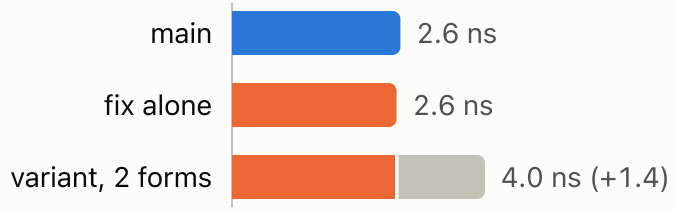
Memory 43.9 → **32.05 B/record (-27%)**, 32.0-32.1 on every snapshot; 472 records per high record on average. TRecLow is 32 B and 32-byte aligned. **Counter-example**: on the load actor's keys (end-to-end test: few tablets, one channel, consecutive Steps) main shares its high parts well and takes 28.0 B per item against 32.0.

Two forms behind a variant: about +1 ns per Next

Next, cold scan, ns per record; grey = variant dispatch



Next, hot: the SST in L2, ns per record



per record	main	fix alone	2 forms
instructions / Next	32	34	48
mispredicts / Next	0.23	0.006	0.021
merge, ns	22.4	21.5	23.2
Get cold, µs	3.62	2.78	3.39

- TLogoBlobSstIndex holds TwoLevel or Packed in a variant; its iterator switches on the form.
- Over the fix alone: hot Next +1.4 ns, cold +1.0 ns, +14 instructions per call; not mispredicts.
- Merge 23.2 ns, about main's. A cold Get pays +22% over the fix alone for building the iterator, and is still faster than main.

One snapshot (10.6 M records, 272 SSTs), 3 interleaved repeats, identical lookup results. Fix alone = the two-level layout built without the variant; 2 forms = TwoLevel + Packed, as in this PR.

Fixed-width columns: O(1) access by position

Keep the two-level high part; store each low field as a **column of fixed-width bit fields**. Field c of record i sits at a computable bit position: no sequential decoding, the iterator is the two-level one with an index instead of a pointer.

Dict

PartCrc, BlobSize, Ingress, TypeId, Size: few distinct values per SST; store the index into the SST's sorted distinct values.

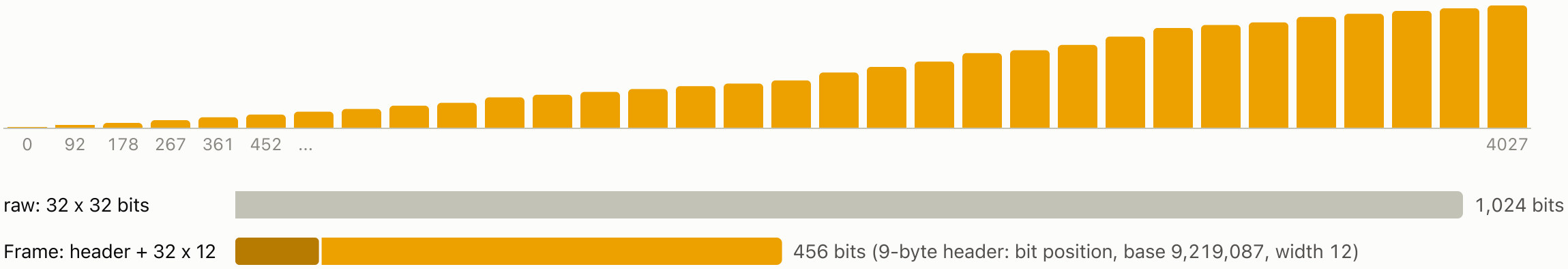
BlobSize 9.2 bits (raw 26), Ingress 12.0 (raw 64), PartCrc 2.3 (raw 6), dictionaries included.

Frame

Step, Cookie, Offset: change little between neighbours; per 32 records a base (the minimum) and a bit width, each value stored as value - base.

Step 17.7 bits (raw 32; as a Dict 78), Cookie 6.0 (raw 24), frame headers included.

32 records of one frame (record 16,416 … 16,447 of the SST): Step - base



Bits per record over all 5 snapshots. Memrec fields (TypeId, Offset, Size) are synthesised in the snapshots, so their survey is left out.

Before and after: twelve records in Packed

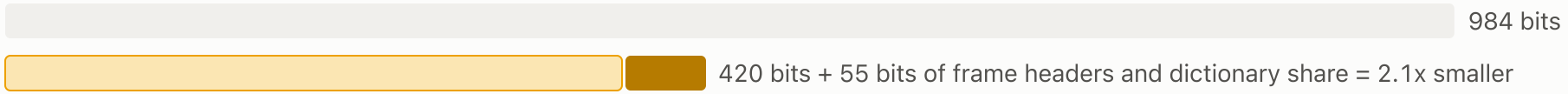
Records 4,044 … 4,055 of the largest SST of one snapshot. Each column holds one field of every record; record i of a column sits at its start + i x width.

BlobSize → Dict index 26 → 10 bits			Step → offset from the frame base 32 → 20 bits			Cookie → offset 24 → 5 bits	
1 MiB	#255	0 0 1 1 1 1 1 1 1 1	24,555,760	+124,701	0 0 0 1 1 1 1 0 0 1 1 1 0 0 0 1 1 1 0 1	0	0 0 0 0 0
192 KiB	#47	0 0 0 0 1 0 1 1 1 1	24,560,731	+129,672	0 0 0 1 1 1 1 1 1 0 1 0 1 0 0 0 1 0 0 0	15	0 1 1 1 1
192 KiB	#47	0 0 0 0 1 0 1 1 1 1	24,569,248	+138,189	0 0 1 0 0 0 0 1 1 0 1 1 1 1 0 0 1 1 0 1	4	0 0 1 0 0
1,412 KiB	#352	0 1 0 1 1 0 0 0 0 0	24,590,353	+159,294	0 0 1 0 0 1 1 0 1 1 1 0 0 0 1 1 1 1 1 0	0	0 0 0 0 0
256 KiB	#63	0 0 0 0 1 1 1 1 1 1	24,594,601	+163,542	0 0 1 0 0 1 1 1 1 1 1 0 1 1 0 1 0 1 1 0	13	0 1 1 0 1
256 KiB	#63	0 0 0 0 1 1 1 1 1 1	24,599,762	+168,703	0 0 1 0 1 0 0 1 0 0 1 0 1 1 1 1 1 1 1 1	19	1 0 0 1 1
256 KiB	#63	0 0 0 0 1 1 1 1 1 1	24,602,191	+171,132	0 0 1 0 1 0 0 1 1 1 0 0 0 1 1 1 1 1 0 0	3	0 0 0 1 1
1,792 KiB	#446	0 1 1 0 1 1 1 1 1 0	24,608,080	+177,021	0 0 1 0 1 0 1 1 0 0 1 1 0 1 1 1 1 1 0 1	0	0 0 0 0 0
212 KiB	#52	0 0 0 0 1 1 0 1 0 0	24,612,906	+181,847	0 0 1 0 1 1 0 0 0 1 1 0 0 1 0 1 0 1 1 1	9	0 1 0 0 1
128 KiB	#31	0 0 0 0 0 1 1 1 1 1	24,617,467	+186,408	0 0 1 0 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0	20	1 0 1 0 0
256 KiB	#63	0 0 0 0 1 1 1 1 1 1	24,618,546	+187,487	0 0 1 0 1 1 0 1 1 1 0 0 0 1 0 1 1 1 1 1	14	0 1 1 1 0
256 KiB	#63	0 0 0 0 1 1 1 1 1 1	24,628,939	+197,880	0 0 1 1 0 0 0 0 0 1 0 0 1 1 1 1 0 0 0	2	0 0 0 1 0

BlobSize dictionary of the SST
sorted, 818 values x 8 B

#31	131,072
#47	196,608
#52	217,088
#63	262,144
#255	1,048,576
#352	1,445,888
#446	1,835,008

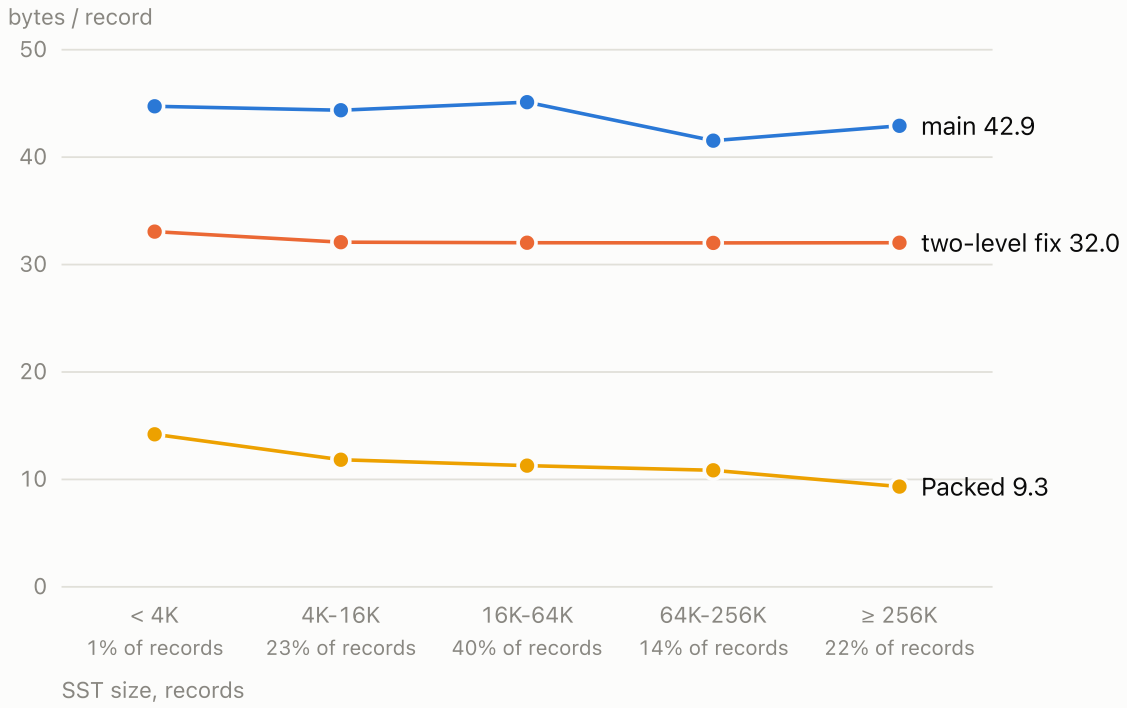
frame of records 4,032-4,063
Step: base 24,431,059, width 20
Cookie: base 0, width 5
9-byte header per frame and column: bit position, base, width



Computed as TPackedLogoBlobSstIndex::PackDict / PackFrame do: Dict index = position in the SST's sorted distinct values, width = bits of (count - 1); Frame = 32 records by position, base = minimum, width = bits of (max - min). The Step frame jumps by ~0.6 M at its end, hence 20 bits (17.7 on average over all snapshots). Any record's value is one shift and mask away.

Packed is smaller than main in every SST; small SSTs pay more per record

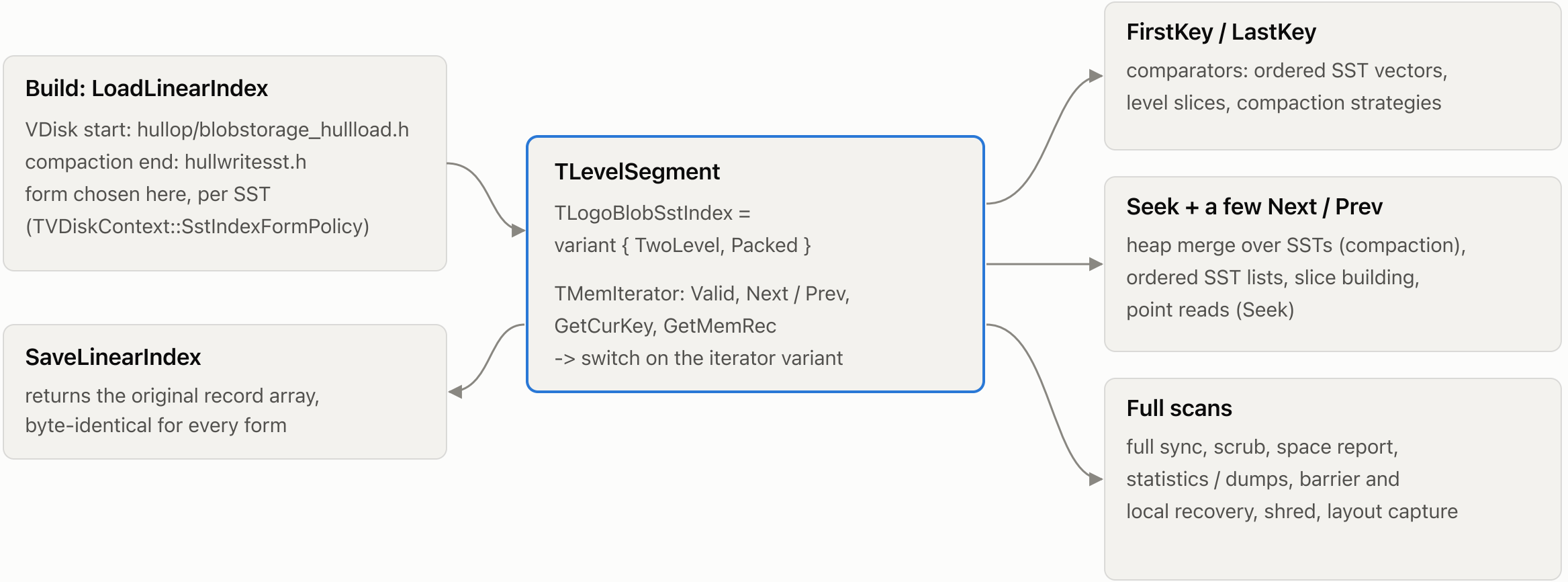
Index bytes per record by SST size (record-weighted)



- 4,212 SSTs, median 13,674 records (p90 48,323, max 1.37 M).
- The two-level forms are flat. Packed grows below ~4 K records (14 B/record: dictionaries and frame headers), but those SSTs hold < 1% of the records.
- Larger than main: the two-level fix in 126 SSTs (all ≤ 67,661 records), **Packed in none**.
- Packed 10.9 B/record over all snapshots; 5.8-10.9 B depending on the synthesised memrecs.

5 production index snapshots of block-storage clusters (block-4-2), 4,212 SSTs, 109.7 M records; memrecs synthesised; production huge-blob threshold.

Nothing above TLevelSegment knows the form



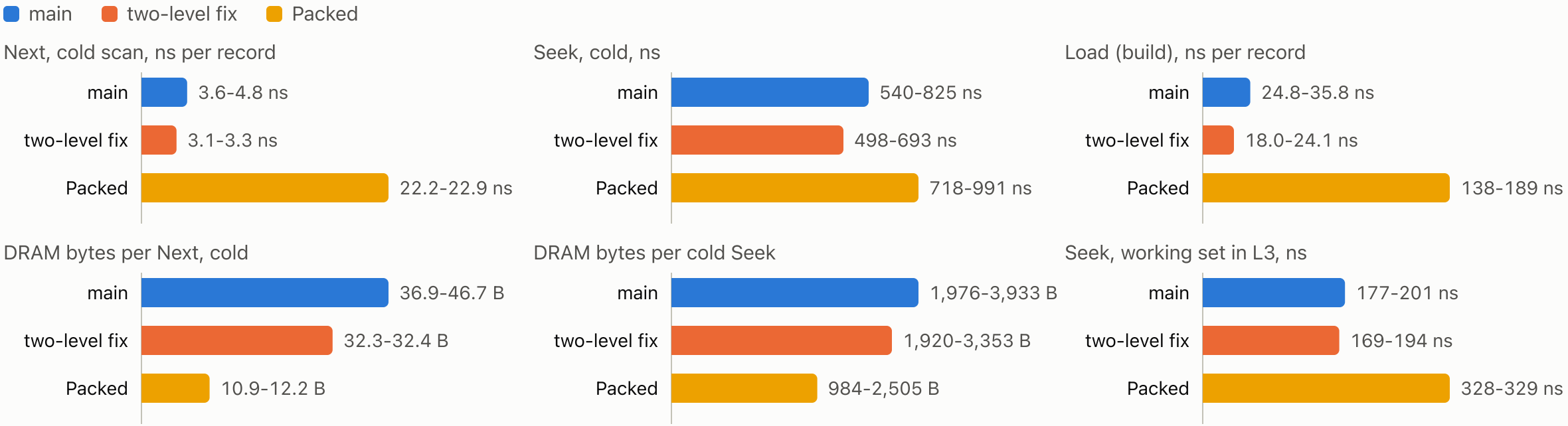
The policy comes from the EnableVDiskPackedSstIndex feature flag, latched at VDisk start (restart required). Switching it either way is safe: in-memory indexes are rebuilt from the unchanged on-disk records.

Each benchmark stands for a production use, with known distortions

benchmark	stands for	caveat
load: build every SST of the snapshots; bytes from the SstIndex memory counter, Save checked byte for byte	memory of a VDisk; VDisk start, end of compaction	**memrecs synthesised** from the keys: Packed 5.8-10.9 B/record depending on them
Next / Seek, cold (every SST once in random order, index >> L3) and hot (in L3 / L2)	full scans (sync, scrub, compaction input), point reads	no work per record: an upper bound on the decode share
Get: Seek in every SST of one VDisk whose range holds the key (~21)	the index part of a read	no fresh segment, barriers, query layer or disk
merge: heap merge of an SST with up to 7 overlapping SSTs of its VDisk	a compaction step	no memrec merge, no disk writes
neighbour: pointer chase on another core sharing the L3 while the bench scans	other VDisks and tenants on the L3	synthetic neighbour; the bench does nothing but decode
end to end: one ydbd node, block-4-2 over 8 NVMe partitions, ~96 M part records written by the load actor	startup, RSS, latency, CPU of a running VDisk	**load-actor keys** favour main and over-compress for Packed

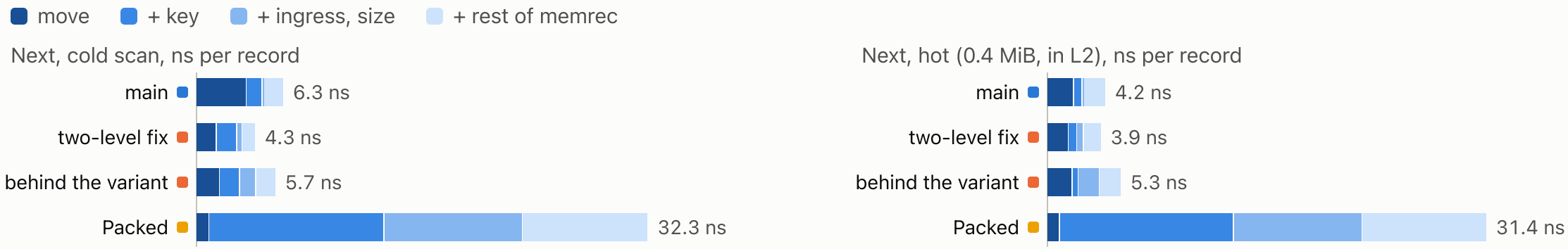
Data: 5 production index snapshots of block-storage clusters (block-4-2), 4,212 SSTs, 109.7 M records; memrecs synthesised.
Micro and operation benchmarks: AMD EPYC 7702 (Zen 2), one core of a 16 MiB L3, Oct 22 2026, median of 3; two-level fix built without the form variant; Packed = an earlier revision of this PR. All builds return identical lookup results.

Packed: +18 ns per Next, 1.2-1.3x per cold Seek



AMD EPYC 7702 (Zen 2), one core of a 16 MiB L3, Oct 22 2026, median of 3; two-level fix built without the form variant; Packed = an earlier revision of this PR. Ranges = two snapshots; Load: all 5. Behind this PR's variant the two-level fix scans at 3.9-4.2 ns per Next (section 3).

Moving vs decoding: Packed moves for free and pays ~31 ns to decode



- **Row layouts**: moving costs ~1.5-2 ns (main cold 3.6: memory-bound), decoding the whole record 2.7-4.0 ns. Behind the variant the dispatch is paid per getter call: +1.5 ns on the full record.
- **Packed**: Next is an index increment (0.9 ns); key +12.5, memrec +18.9 ns, cold or hot alike: pure compute.

Seek, ns	move only	full record
main cold / hot	555 / 128	546 / 129
two-level fix cold / hot	515 / 123	500 / 124
Packed cold / hot	445 / 181	706 / 205

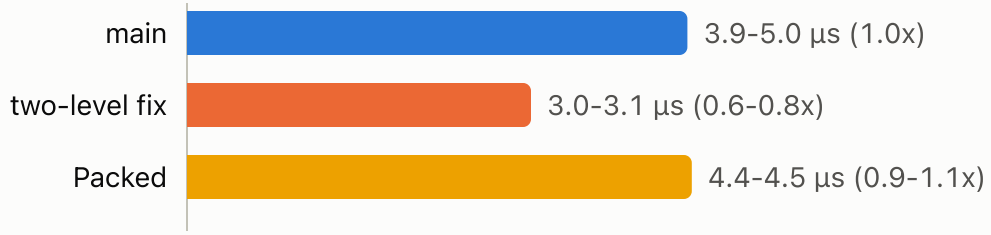
Reading the found record is within noise except for Packed: +261 ns cold (its memrec columns sit in other cache lines than the key columns the search touched), +24 ns hot.

Oct 23 2026, one snapshot, median of 3. "Behind the variant" = the two-level layout in the two-form variant: this PR's default. Hot sets are sized in MiB of index, so forms walk different SSTs: compare within a form.

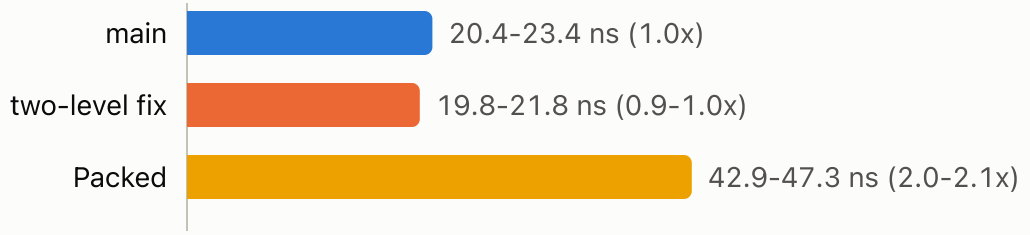
A Get costs 0.9-1.1x main with Packed; a compaction step 2.0-2.1x

main two-level fix Packed

Get: Seek in every SST of the VDisk holding the key (~21 SSTs), µs



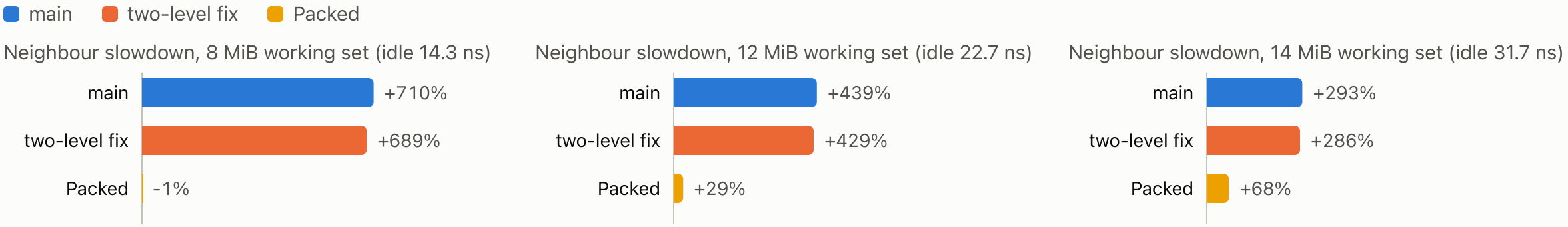
Compaction step: heap merge of ~8 SSTs, ns per output record



- A Get seeks ~21 SSTs of one VDisk: level 0 plus every level whose key range holds the key.
- Packed reads 3.1-3.6x fewer DRAM bytes per Get than main, which mostly pays for its decoding.
- A compaction step decodes every record of every input: Packed 2.0-2.1x main.
- The two-level fix alone is 0.9-1.0x main; behind the variant its merge is 21.1-23.8 ns, about main's.

AMD EPYC 7702 (Zen 2), one core of a 16 MiB L3, Oct 22 2026, median of 3; two-level fix built without the form variant; Packed = an earlier revision of this PR. Get: 200 K keys, half present, half near misses; merge: 300 jobs.

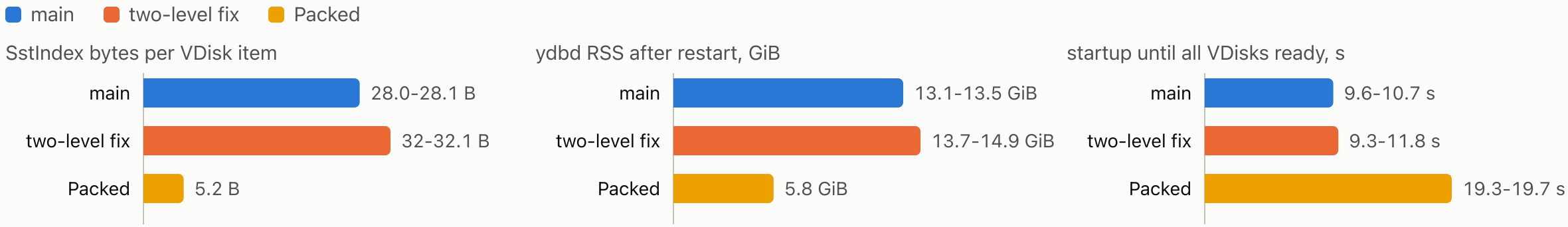
Row layouts evict a neighbour's cache; Packed barely does



- The row layouts stream 11.1-11.7 GB/s (two-level fix) to 14-15 GB/s (main) through the L3 and push out all of the neighbour's data: its loads take DRAM latency at every size.
- Packed streams 0.5 GB/s: harmless up to 8 MiB, but its columns and dictionaries live in the L3 and cost a near-full neighbour +29% / +68%.
- Mostly a rate effect: per second Packed moves ~28x fewer bytes than main.

Neighbour: random pointer chase over 8 / 12 / 14 MiB on another core sharing the 16 MiB L3, while the bench scans one snapshot cold; slowdown = neighbour ns per load vs idle, median of 3. Other load shared the L3 during part of the run.

One ydbd: RSS 2.3x smaller, startup 1.9x longer with Packed



	main	two-level fix	Packed
read p50, ms	1.4-2.4	1.6-1.8	2.3-2.7
read p99, ms	74-117	35-53	25-58
CPU during session, cores	18-25.1	26.4-28.5	26.4-31.2

startup	measured vs main	96 M x extra Load ns
two-level fix	+0.4 s	-1.2 s
Packed	+9.4 s	+11.8 s

Startup grows by records x build time. Build per record: two-level 19 ns, Packed 154 ns; the largest snapshot VDisk (5.5 M records) builds in 0.11 / 0.85 s. Compaction pays the same build per written SST.

One node, block-4-2 over 8 NVMe partitions, ~96 M part records written by the load actor; two restart + 5-minute sessions per form (closed loop, 64 reads in flight: compare medians). Two-level = the form behind the two-form variant. Load-actor keys: main beats the fix here, and Packed compresses better than on production keys (5.2 vs 10.9 B).

Proposal: merge with Packed off by default

two-level fix: the default form

-27% on the production snapshots (43.9 → 32.1 B/record), cold scans and Gets faster than main even behind the variant.

+14% on load-actor keys; hot Next 4.0 vs main's 2.6 ns with the variant.

Packed: behind EnableVDiskPackedSstIndex

-75% (10.9 B/record), Packed smaller than main in every SST; 2.3x smaller RSS end to end.

Costs ~18 ns per Next, 1.2-1.3x on cold seeks, 2.0-2.1x per merged record, 1.9x startup; a Get is 0.9-1.1x main.

Before enabling Packed by default

1. **Real memrecs**: Packed is 5.8-10.9 B/record depending on them; a snapshot with real memrecs turns the range into one number.
2. **Production-like keys end to end**: the load actor's keys favour main and over-compress for Packed.
3. **Startup and full sync of a full node**: build time adds to restart; a full sync iterates the sender's whole index (~1.3 s per peer for 45 M records, two-level ~0.3 s).
4. **Dispatch**: ~1 ns per Next and ~20% of a cold Get with two forms; choosing the form once per SST would remove both.
5. **Oversized SSTs**: the packed stream uses 32-bit bit positions and aborts above ~23 M records in the worst case (default SSTs hold up to ~3 M); a fallback to two-level is not implemented.